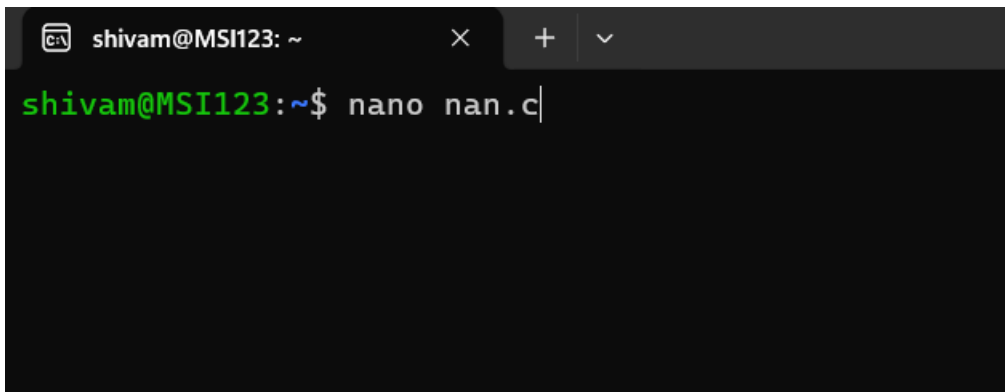


WEEK 2

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

1) Create a file

A terminal window with a dark background. The title bar shows 'shivam@MSI123: ~' and window control buttons. The prompt is 'shivam@MSI123:~\$' and the command 'nano nan.c' is being typed.

Then press enter to open command

2)write a code

```
shivam@MSI123: ~
GNU nano 7.2 nan.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());

        execl("/bin/sh", "sh", "-c", command, NULL);

        // Executes only if execl fails
        perror("Execution failed");
        exit(1);
    }

    else {
        // Parent Process
        printf("\nParent Process\n");

        [ Read 46 lines ]
    }
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^G Location   ^M-U Undo     ^M-A Set Mark ^M-] To Bracket
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_/ Go To Line ^M-E Redo     ^M-G Copy     ^_ Where Was
```

```
    else {
        // Parent Process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

- 2) To save press control O
- 3) To come back to shell press control X
- 4) Then for compilation write a command
gcc filename.c -o filename

```
shivam@MSI123: ~
shivam@MSI123: ~$ nano nan.c
shivam@MSI123: ~$ gcc nan.c -o nan|
```

Then press enter then to view a output write a command
./filename

```
shivam@MSI123:~$ nano nan.c
shivam@MSI123:~$ gcc nan.c -o nan
shivam@MSI123:~$ ./nan
Enter Linux command: |
```

Then you will be asking to enter linux command to view the output
In ENTER LINUX COMMAND: enter either "pwd" command (or) "ls" command
to see the output of your code

```
shivam@MSI123:~$ nano nan.c
shivam@MSI123:~$ gcc nan.c -o nan
shivam@MSI123:~$ ./nan
Enter Linux command: pwd

Parent Process
Parent PID: 451

Child Process
Child PID: 474
/home/shivam
Child process completed.
shivam@MSI123:~$ |
```

OUTPUT :

Enter Linux command : pwd

Parent Process
Parent PID: 474

Child Process
Child PID: 474
/home/shivam
Child process completed.

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
shivam@MSI123:~$ uname
Linux
shivam@MSI123:~$ |
```

```
shivam@MSI123:~$ uname
Linux
shivam@MSI123:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       12
On-line CPU(s) list:         0-11
Vendor ID:                    GenuineIntel
Model name:                   12th Gen Intel(R) Core(TM) i5-12450H
CPU family:                   6
Model:                        154
Thread(s) per core:           2
Core(s) per socket:           6
Socket(s):                    1
Stepping:                     3
BogoMIPS:                     4992.00
Flags:                        fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtsc
                                cp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid
                                sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs i
                                bpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflush
                                opt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_
                                clear serialize ibt flush_l1d arch_capabilities

Virtualization features:
Virtualization:               VT-x
Hypervisor vendor:            Microsoft
Virtualization type:          full
Caches (sum of all):
L1d:                           288 KiB (6 instances)
L1i:                           192 KiB (6 instances)
L2:                             7.5 MiB (6 instances)
L3:                             12 MiB (1 instance)
NUMA:
NUMA node(s):                  1
NUMA node0 CPU(s):             0-11
Vulnerabilities:
Gather data sampling:          Not affected
Ghostwrite:                    Not affected
Indirect target selection:      Not affected
Itlb multihit:                 Not affected
```

```
Virtualization features:
Virtualization:               VT-x
Hypervisor vendor:            Microsoft
Virtualization type:          full
Caches (sum of all):
L1d:                           288 KiB (6 instances)
L1i:                           192 KiB (6 instances)
L2:                             7.5 MiB (6 instances)
L3:                             12 MiB (1 instance)
NUMA:
NUMA node(s):                  1
NUMA node0 CPU(s):             0-11
Vulnerabilities:
Gather data sampling:          Not affected
Ghostwrite:                    Not affected
Indirect target selection:      Not affected
Itlb multihit:                 Not affected
L1tf:                           Not affected
Mds:                           Not affected
Meltdown:                      Not affected
Mmio stale data:               Not affected
Old microcode:                 Not affected
Reg file data sampling:         Vulnerable: No microcode
Retbleed:                      Mitigation; Enhanced IBRS
Spec rstack overflow:          Not affected
Spec store bypass:             Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:                    Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:                    Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI BHI_DIS_S
Srbds:                          Not affected
Tsa:                           Not affected
Tsx async abort:               Not affected
Vmscape:                       Not affected
shivam@MSI123:~$
```

```
shivam@MSI123:~$ ps
  PID TTY          TIME CMD
  297 pts/0        00:00:00 bash
  589 pts/0        00:00:00 ps
shivam@MSI123:~$
```

```
shivam@MSI123:~$ top
top - 05:52:22 up 19 min, 1 user, load average: 0.02, 0.02, 0.00
Tasks: 25 total, 1 running, 24 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.6 us, 0.2 sy, 0.0 ni, 98.3 id, 0.6 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 7794.6 total, 7205.1 free, 560.9 used, 183.0 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7233.7 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
613	root	20	0	13420	8920	6940	S	0.7	0.1	0:00.02	wget
1	root	20	0	21516	13000	9676	S	0.0	0.2	0:00.51	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.00	init-systemd(Ub
6	root	20	0	3196	2132	2008	S	0.0	0.0	0:00.00	init
52	root	19	-1	50360	15972	14860	S	0.0	0.2	0:00.19	systemd-journal
102	root	20	0	25272	6652	5164	S	0.0	0.1	0:00.48	systemd-udevd
108	systemd+	20	0	21460	13216	10960	S	0.0	0.2	0:00.08	systemd-resolve
109	systemd+	20	0	91028	7964	6996	S	0.0	0.1	0:00.07	systemd-timesyn
173	root	20	0	4244	2680	2432	S	0.0	0.0	0:00.00	cron
174	message+	20	0	9592	5404	4712	S	0.0	0.1	0:00.10	dbus-daemon
182	root	20	0	17968	8852	7812	S	0.0	0.1	0:00.09	systemd-logind
190	root	20	0	1755848	13248	10888	S	0.0	0.2	0:00.13	wsl-pro-service
202	syslog	20	0	222516	5920	4544	S	0.0	0.1	0:00.07	rsyslogd
208	root	20	0	3124	1972	1836	S	0.0	0.0	0:00.00	agetty
222	root	20	0	107020	23212	13744	S	0.0	0.3	0:00.09	unattended-upgr
295	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
296	root	20	0	3200	1248	1108	S	0.0	0.0	0:00.04	Relay(297)
297	shivam	20	0	6076	5384	3668	S	0.0	0.1	0:00.05	bash
298	root	20	0	6704	4712	3928	S	0.0	0.1	0:00.01	login
341	shivam	20	0	20312	11572	9448	S	0.0	0.1	0:00.06	systemd
342	shivam	20	0	21156	3588	1860	S	0.0	0.0	0:00.00	(sd-pam)
366	shivam	20	0	6080	5240	3676	S	0.0	0.1	0:00.00	bash
592	shivam	20	0	9300	5696	3512	R	0.0	0.1	0:00.00	top